Catálogo de Desafíos de Valor Agregado — 2.º Parcial **Materia:** Programación I / II **Carrera:** Tecnicatura Superior en Desarrollo de Software — 2.º Año **Instituto:** Instituto Roque González --- ## 📌 Modalidad de Evaluación y Reglas - **Base Obligatoria:** Todos los alumnos/equipos deben tener funcionando el sistema base con la arquitectura modular. - **Modalidad Individual:** Deben elegir e implementar **al menos uno** de los desafíos del **NIVEL 1**. - **Modalidad en Parejas / Duplas (Requisito Obligatorio):** Para rendir de a dos personas, es **obligatorio** elegir e implementar **al menos uno** de los desafíos del **NIVEL 2**. Ambos integrantes deben conocer y defender la totalidad del código. --- ## 🟢 NIVEL 1: Desafíos para Alumnos Individuales *(Foco en algoritmos, análisis de datos, colecciones y reglas de negocio)* ### 1. Búsqueda e Historial por Patente - **Descripción:** Agregar una nueva opción al menú principal que permita ingresar una patente y consulte. - **Salida esperada:** Informa cantidad total de ingresos históricos del vehículo, dinero total acumulado que abonó y promedio de minutos por estadía. - **Conceptos evaluados:** Búsqueda en colecciones, lectura y consolidación de múltiples archivos CSV, cálculo de promedios. ### 2. Reporte Estadístico y Métricas del Negocio - **Descripción:** Al finalizar el procesamiento del lote (Opción 2), el sistema imprime un reporte ejecutivo con métricas avanzadas: - Categoría de vehículo más rentable (mayor recaudación en $). - Porcentaje de vehículos que contrataron servicio de lavado. - Estadía promedio (en minutos) de cada categoría de vehículo. - **Conceptos evaluados:** Acumuladores, conteos condicionales, agregaciones sobre diccionarios y formateo numérico. ### 3. Ordenamiento Algorítmico (Ranking en CSV) - **Descripción:** Generar un archivo adicional denominado `liquidacion_ranking.csv` donde los registros procesados aparezcan ordenados de mayor a menor según el `total_cobrado` (o según la duración en minutos). - **Conceptos evaluados:** Algoritmos de ordenamiento (burbuja, inserción o `sorted()` con funciones lambda / claves de ordenamiento personalizadas). ### 4. Tratamiento de Medianoche (Turno Noche) - **Descripción:** Si en una estadía del mismo día la hora de salida es menor numéricamente a la de entrada (ejemplo: entrada `23:00` y salida `02:30`), en vez de descartar el registro, el sistema debe asumir que cruzó la medianoche y calcular los minutos reales transcurridos (en el ejemplo: 3 horas y media = 210 min). - **Conceptos evaluados:** Lógica condicional avanzada, aritmética modular de tiempo (cruce de las 24:00 hs). ### 5. Simulación de Promociones (Happy Hour o Cliente Frecuente) - **Descripción:** Incorporar una regla de beneficio comercial: - *Opción A (Franja horaria):* Si el ingreso fue antes de las 08:00 o después de las 16:00, se aplica un 15% de descuento sobre el costo base de estacionamiento. - *Opción B (Cliente fiel):* Si la patente ya tiene un registro previo en el historial, se bonifica un 10% del total. - **Conceptos evaluados:** Reglas de negocio dinámicas, anidamiento de descuentos y trazabilidad. ### 6. Exportación Filtrada por Categoría - **Descripción:** Agregar una opción de reporte que permita exportar tres archivos CSV separados: `liquidacion_autos.csv`, `liquidacion_motos.csv` y `liquidacion_camionetas.csv`, cada uno con su propio encabezado y una fila final de subtotales exclusivos para esa categoría. - **Conceptos evaluados:** Filtrado de colecciones, generación de múltiples archivos de salida y funciones reutilizables de escritura. ### 7. Detección de Estadías Excesivas (Alertas de Control) - **Descripción:** Analizar los movimientos para detectar vehículos que hayan permanecido más de 10 horas en el mismo día. El sistema debe emitir una advertencia en pantalla y registrar los casos en un archivo especial `alertas_seguridad.csv`. - **Conceptos evaluados:** Detección de anomalías en datos, filtros de seguridad y persistencia de eventos. ### 8. Paginador de Resultados en Consola - **Descripción:** Agregar una función para visualizar en consola las liquidaciones en una tabla formateada en ASCII, mostrando de a 5 o 10 vehículos por página y requiriendo presionar `[ENTER]` para ver el siguiente bloque de registros. - **Conceptos evaluados:** Rebanado de listas (slicing), control de flujo interactivo y presentación limpia en interfaz de texto. --- ## 🔵 NIVEL 2: Desafíos para Duplas / Parejas *(Foco en arquitectura de software, bases de datos relacionales, interfaces gráficas y persistencia avanzada)* ### 1. Persistencia Relacional con Base de Datos (`sqlite3`) — *Recomendada* - **Descripción:** Utilizando la biblioteca nativa `sqlite3` de Python (no requiere instalar paquetes externos), crear una base de datos local `autopark.db` con tablas relacionales (por ejemplo: `liquidaciones` y `tarifas`). - **Funcionalidad:** - Migrar los registros validados a la tabla SQLite. - Agregar una opción al menú para consultar métricas mediante sentencias SQL con agrupamiento (`SELECT

tipo_vehiculo, COUNT(*), SUM(total_cobrado) FROM liquidaciones GROUP BY tipo_vehiculo`). - **Conceptos evaluados:** SQL en Python, DDL (CREATE TABLE), DML (INSERT/SELECT), manejo de cursores y conexiones. ### 2. Interfaz Gráfica de Usuario (GUI con `tkinter`) - **Descripción:** Utilizando el módulo nativo `tkinter` de Python, desarrollar una ventana gráfica de escritorio para el módulo de "Cobro Manual en Ventanilla". - **Componentes:** - Cajas de texto para patente y horarios. - Menú desplegable para el tipo de vehículo. - Checkbox para solicitar servicio de lavado. - Botón "Liquidar y Guardar" que calcule en tiempo real, muestre un cartel flotante con el total y registre la operación en el CSV correspondiente. - **Conceptos evaluados:** Programación orientada a eventos, widgets gráficos, variables de control y layouts. ### 3. Control Dinámico de Capacidad y Cocheras (Cupos) - **Descripción:** Agregar en `tarifas.json` un límite máximo de cocheras simultáneas por categoría (ejemplo: 10 autos, 5 motos, 3 camionetas). - **Funcionalidad:** - Al procesar el lote, el sistema debe verificar minuto a minuto si hay lugar disponible. - Si un vehículo ingresa en un horario donde todas las cocheras de su tipo estaban ocupadas por otros autos no retirados, el movimiento se rechaza con el motivo `"Sin cochera disponible"`. - **Conceptos evaluados:** Intervalos de concurrencia, gestión de estado temporal y modelado de inventario/capacidad. ### 4. Generador de Tickets Fiscales Digitales (`comprobantes.py`) - **Descripción:** Crear un módulo que emita un archivo de texto individual (`TKT-0001.txt`, `TKT-0002.txt`, etc.) por cada operación manual registrada o por cada movimiento del lote. - **Formato del ticket:** - Encabezado de fantasía (Razón social, CUIT, Dirección). - Número de comprobante correlativo con ceros a la izquierda. - Desglose discriminado: Estacionamiento base, Bonificación por lavado, Servicio de lavado, Subtotal, IVA (21%) y Total final. - **Conceptos evaluados:** Formateo avanzado de cadenas (f-strings con alineación), numeración secuencial persistente y exportación en formato plano. ### 5. Sistema de Usuarios, Roles y Seguridad (`usuarios.json`) - **Descripción:** Implementar una pantalla de Login por consola antes de acceder al menú del sistema, utilizando contraseñas con hash (`hashlib`, estándar de Python). - **Roles:** - `Operador`: solo tiene acceso a la Opción 1 (Cobro manual en ventanilla). - `Administrador`: tiene acceso a todo el sistema, incluyendo procesamiento de lotes y reportes contables. - **Trazabilidad:** Cada registro guardado en los CSVs debe incluir una columna con el nombre del operador que realizó la operación. - **Conceptos evaluados:** Autenticación, hashing criptográfico de contraseñas, control de acceso basado en roles (RBAC) y auditoría. ### 6. Arqueo y Cierre de Caja de Tesorería - **Descripción:** Implementar un flujo completo de caja: - Al iniciar el programa, el operador ingresa el "Fondo Inicial de Caja" en efectivo. - En cada cobro manual, se consulta el medio de pago: `Efectivo` o `Transferencia / Tarjeta`. - Opción de "Cierre de Caja": totaliza el dinero recaudado en efectivo, el dinero digital, calcula el saldo teórico que debe haber en la caja física y genera un reporte final `cierre_caja.json`. - **Conceptos evaluados:** Modelado de estados contables, serialización JSON y control de tesorería. ### 7. Reporte HTML Interactivo (Dashboard Web sin frameworks) - **Descripción:** Generar un archivo `reporte_diario.html` que pueda abrirse directamente en cualquier navegador web. - **Características:** - Diseño visual con CSS embebido (tarjetas con totales recaudados, promedios y porcentajes). - Tabla con colores alternados según tipo de vehículo e indicadores visuales (verde si pagó lavado, gris si fue estándar). - **Conceptos evaluados:** Generación dinámica de código HTML/CSS desde estructuras de datos en Python. ### 8. Tarifario Dinámico por Día y Feriados (`datetime`) - **Descripción:** Utilizando el módulo `datetime`, el sistema debe interpretar la fecha de operación (`fecha_operacion` en el JSON) y determinar el día de la semana. - **Regla:** - Si es fin de semana (sábado o domingo) o feriado configurado, se aplica un recargo automático establecido en `tarifas.json` (+20% estacionamiento, +10% lavado). - La función recursiva de cálculo debe adaptarse para recibir el multiplicador o tarifa correspondiente a la fecha. - **Conceptos evaluados:** Manipulación avanzada de fechas con `datetime`, parametrización y adaptación de funciones recursivas.